CEPHALOPOD AR · DESIGN RATIONALE

Designing an Infinite Ocean for a Phone

What we built, why it had to be this way,
and what is genuinely new about it

The Procedural Terrain Subsystem
Marine Biology AR Application
Architecture through the World Context API

This is a design document, not an implementation reference. We stay out of the code. For each part of the system we answer three questions — *what* we built, *why* it had to be built that way, and *how* it works in principle — and we then say plainly which parts we think are genuinely novel and which are our competent applications of somebody else's technique.

Contents

A note on how novelty is graded

Every claim of novelty we make carries an explicit grade. We grade them because overclaiming is the fastest way to lose a reviewer, and because most of what our system does is — correctly — standard technique that we did not invent. Our three grades are:

Grade	Means	Standard of evidence
★★★ NEW	We are not aware of prior work doing this	We expect it to survive a literature check, and we would defend it as a contribution in its own right
★★ COMBINATION	We used known techniques, but our assembly, framing or application of them is new	We would defend it as a systems or application contribution, not as a new algorithm

★	STANDARD	Established technique that we applied competently	We cite it rather than claim it. Listed here for completeness

Roughly two thirds of what we built is grade ★. We say so plainly because it is what makes our grade ★★★ claims worth reading.

1 The Problem We Were Solving

> We needed an ocean for a marine biology lesson. Not a photograph of one, and not one ocean — a different one for the reef lesson, the kelp forest lesson, and the deep canyon lesson. On a phone. In the room the student is standing in. Built by a teacher who does not write code.

That paragraph contains four constraints, and almost every decision we made is downstream of one of them. We state them precisely before describing anything we built, because several of our choices look arbitrary until you know which constraint forced them.

1.1 The four constraints

Constraint	What it rules out	What it forces
C1 — Many environments One reef is not a curriculum	Hand-authored scenes. Every new biome would be an artist-week and a new app build	Environments must be *data*, generated at runtime, not scenes shipped in the binary
C2 — A phone Mid-range Android and iOS	Large meshes, high draw calls, heavy textures, per-frame CPU scene management, shadows on everything	Streaming with level of detail; shared meshes; textureless shaders where possible; work moved off the main thread and onto the GPU
C3 — Augmented reality The ocean sits on a real floor	A world pinned to the origin. The seabed must be at the height of the detected plane, and the world must follow the user as they physically walk	Everything anchors to an AR plane; the world streams around the camera; nothing may assume a fixed world origin
C4 — A teacher authors it No code, no engine, no jargon	Exposing noise octaves, lacunarity, fog density or LOD tables. Also rules out "just give them fewer sliders" — the wrong subset still breaks	A small vocabulary of meaningful choices, and a layer that guarantees no choice can produce a broken world

C1 and C4 pulled hardest on us, and they pull in opposite directions. C1 wants generality: we should be able to express many oceans. C4 wants safety: our user must not be able to express a bad one. Resolving that tension is where we think the most interesting part of our work lives, and it is the subject of section 8.

1.2 Why not the obvious alternatives

We had three simpler options, and we want to say why we rejected each, because "we used procedural generation" is not by itself a justification.

Ship a few hand-made scenes. This is the standard approach and we agree it is genuinely better for quality. We rejected it on C1: each environment costs artist time and app size, and a teacher cannot make a new one. It also fails quietly on C3 — a hand-made scene has a fixed extent, so a student who walks four metres in the real world walks off the edge of our ocean.

Generate one large environment at load time. This is better for C1, and it removes the edge problem if we make it big enough. We rejected it on C2: a world large enough that a walking student never reaches its edge does not fit in a phone's memory, and generating it would give us a load screen measured in tens of seconds.

Stream a world from a server. This removes the memory problem. We rejected it on C2 for a different reason — classroom Wi-Fi — and because it adds an operational cost that an educational tool we give away to schools cannot carry.

What we were left with was this: generate the world procedurally, in pieces, around the user, as they move, on the device, from a small description. Everything else we describe follows from that sentence.

Novelty of the problem framing ★★ COMBINATION

We know procedural terrain is old, and we know mobile procedural terrain is not new. What we think is uncommon is the *conjunction* we had to satisfy: an infinite streamed procedural world, anchored in augmented reality, authored by a domain expert who is not a developer, with a correctness guarantee that their authoring cannot break it. Each of our constraints has been addressed in isolation by prior work; we are not aware of a system that addresses all four together, and it is in the interactions between them — section 8.3 in particular — that we found our genuinely new material.

2 The Architecture in One Page

> We built the subsystem as a pipeline of six stages. Data flows one way. Each stage has a single responsibility and knows nothing about the stages after it — which is what lets us test it in pieces and replace it in pieces.

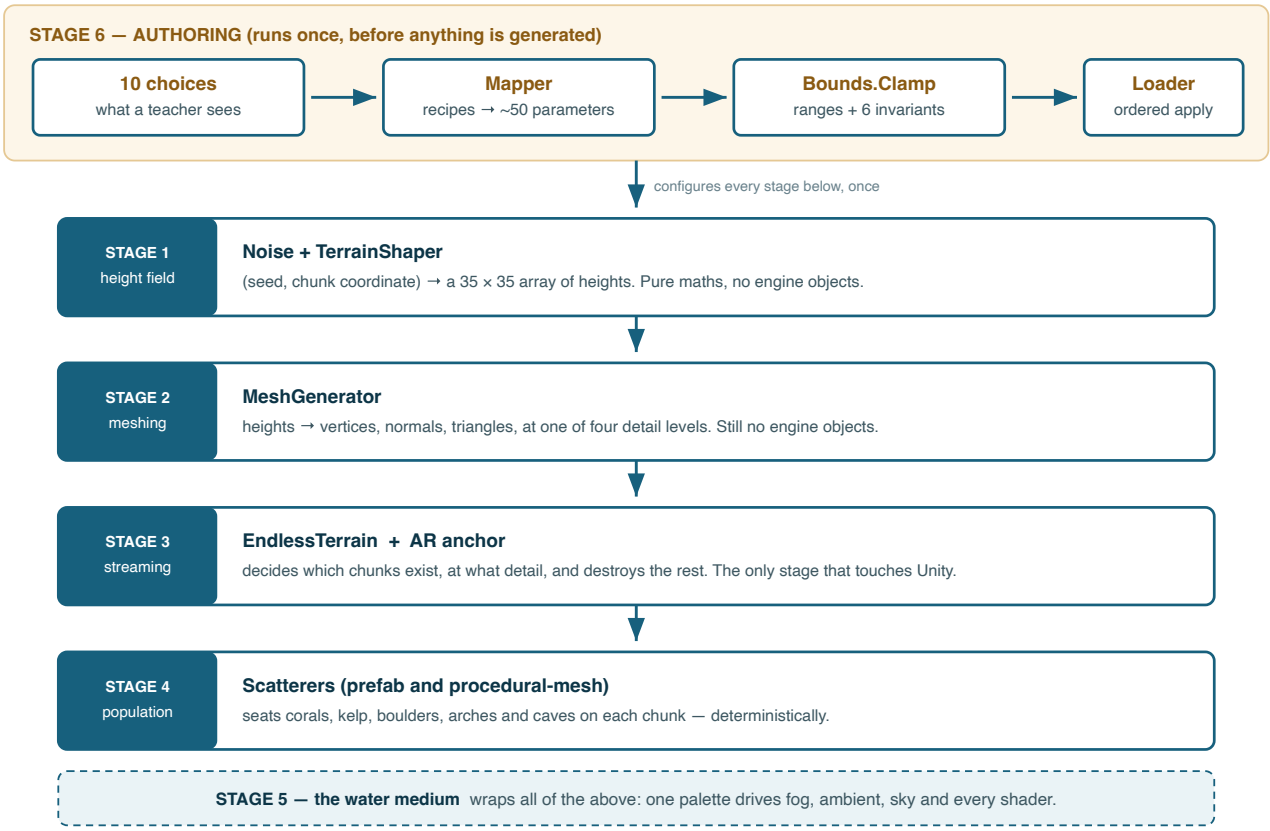

Figure 1 — The six stages. Authoring (stage 6) runs once at load and configures everything below it; stages 1–4 then run continuously as the user moves; stage 5 is not a step in the pipeline but a shared context every drawing stage reads from. We kept the flow strictly one-way so that stages 1 and 2 can run on background threads: they touch no engine state at all.

2.1 What happens when you tap the floor

The clearest way we can explain the architecture is to follow one user action all the way through.

1. **The student points the phone at the floor.** AR Foundation detects a horizontal plane.
2. **They tap it.** A raycast finds the plane, an anchor is created at that exact spot, and the environment prefab is instantiated as a child of that anchor. From this moment the ocean is pinned to a physical location in the room; if tracking corrects itself, the whole world corrects with it.
3. **Our loader reads the chosen environment** — a small file of numbers — clamps it, and pushes those numbers into five systems in a fixed order, finishing with the streamer.
4. **The streamer asks for the chunks around the camera.** Each request goes to a background thread, which generates a height field and turns it into mesh data without touching Unity.
5. **Results come back on the main thread** and become visible geometry. The nearest ring of chunks also gets a collider.

6. **Because a collider now exists, life can be placed.** Each scatterer raycasts down onto the chunk to find the seabed, applies its placement rules, and seats corals and rocks.
7. **The student walks.** New chunks appear ahead, distant ones drop to coarser detail, and chunks far behind are destroyed. Memory stays flat no matter how far they walk.
8. **They walk back.** We regenerate the destroyed reef — *identically*, because we never made it random in the first place. We saved nothing to make this true.

2.2 The two decisions that shaped everything else

We made environments data, not scenes. An environment is about fifty numbers in a small file. It is not a Unity scene, not a prefab, and not a set of assets. This is what let us satisfy C1: a new biome costs a file, not a build. It also let us make our three shipped biomes ordinary — we define them in code as data, on exactly the same footing as anything a teacher makes, so they exercise the same code paths we ship and cannot silently diverge from them.

We kept generation pure, application impure, and the boundary explicit. Stages 1 and 2 are pure functions of (seed, coordinate, parameters). They allocate arrays and do arithmetic. They never call an engine API. We did not do this for style: it is the reason we can run them on a thread pool, the reason identical inputs give us identical output on every device, and the reason we can throw the world away and rebuild it rather than store it.

> **Novelty of the architecture** ★ STANDARD
>
> We claim no novelty for the shape of our pipeline. A staged generate-mesh-stream-populate design with a pure/impure split is well-established practice in games and in the procedural content generation literature, and we adopted it deliberately. Our contributions are in specific stages — the seam guarantee (§3), our determinism-as-persistence-strategy argument (§6), our coupling of the rendering medium to the streaming reach (§7), and above all our authoring layer (§8).

3 Making a Seabed

> **What we built:** a function that turns a seed and a world coordinate into a height, such that neighbouring chunks agree perfectly at their shared edges, and such that the same seed always gives us the same seabed.

3.1 Why noise, and why two styles

We made the seabed a *height field*: for every (x, z) there is exactly one height. We made that trade deliberately and early. A height field is cheap to generate, trivially parallel, compact in memory, and we can turn it into a mesh and a collider without any topology work. Its cost is that it cannot represent an overhang, an arch, or a cave — by definition, since each column has one surface. We accepted that cost and solved caves separately (§6.3), rather than paying for a full volumetric representation everywhere in order to have overhangs in a few places.

We take height from fractal noise: several layers of smooth noise summed, each at higher frequency and lower amplitude than the last. This is the standard construction and we claim nothing for it. What mattered for us is that we needed two very different seabeds — gentle dunes for the reef and kelp lessons, and dramatic canyon walls for a deep-water lesson — and that we wanted both from the same machinery, so our streaming, meshing and collision code has only one thing to deal with.

We therefore layer three operators on top of the same fractal sum to get our *Canyons* style:

- **Domain warping** — before sampling, we push the sample point itself around with a second, much smoother noise field. Straight features become meandering ones. This is what stopped our canyons from looking like scratches on a grid.
- **Ridging** — we fold the noise about its midline, so what was a smooth mid-value contour becomes a sharp crest. We raise the fold to a power to control how knife-edged the crests are.
- **Terracing** — we quantise heights into bands with a sharp but continuous transition between them, which gives us flat mesa tops separated by near-vertical cliff faces.

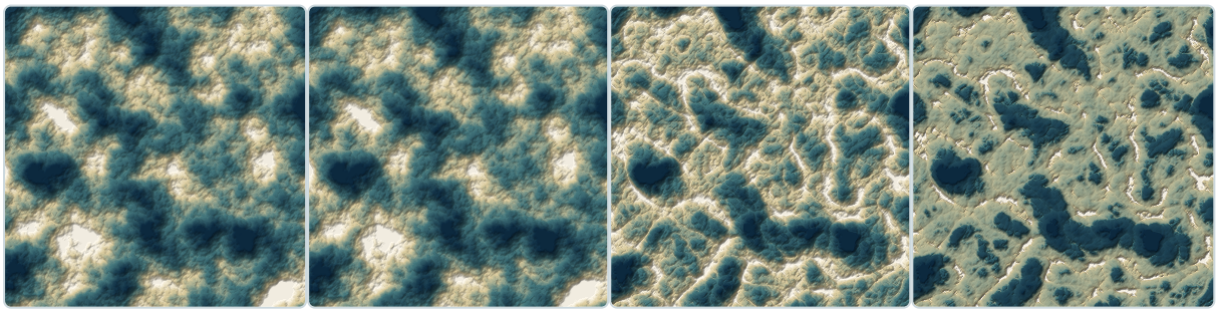

Figure 2 — The three operators, added one at a time. Left to right: the plain fractal sum; with domain warping, which bends features into organic curves; with ridging, which turns smooth contours into crests; with terracing, which flattens the tops and steepens the walls into cliff bands. All four are the same seed and the same noise — only the operators differ. Rendered from a faithful port of the shipping code.

3.2 The seam problem, and why it is a floating-point problem

We generate chunks independently, often on different threads, in no particular order. Two neighbouring chunks share an edge, and every vertex along that edge gets computed *twice* — once by each chunk. If those two computations do not produce bit-identical results, the mesh edges do not line up and a visible seam runs through our world.

What caught us out is that the two computations are algebraically identical and still disagree. Floating-point addition is not associative: adding a large offset to a coordinate before adding a small one loses precision differently than the reverse. A chunk far from the origin has a large coordinate offset, and our noise field has large random per-layer offsets; group them in the wrong order and the shared vertex comes out a few millionths apart in each chunk.

We fixed it by fixing the order of operations: we group and sum the small local terms first, and add the large offsets last, identically in both chunks. It is a one-line concern that is easy to get wrong, its symptom (a shimmering line) looks like a meshing bug rather than an arithmetic one, and — as far as we can find — it is essentially never written down.

We made a second, related decision for the same reason. Normalising each chunk's heights against its own local minimum and maximum gives better contrast, but the range differs per chunk, so shared vertices normalise differently and our seam comes back. We therefore normalise against a deterministic global bound instead, and our authoring layer *forces* this mode rather than exposing it as a preference. That is an early example of a pattern we use throughout §8: where a setting has a correct value, we remove it from the user rather than default it.

Seam-safe coordinate grouping, stated explicitly ★★ COMBINATION

We did not invent this technique — it is folklore among practitioners. What we offer is documenting it as a correctness property with its cause named. We state it as an invariant — *shared border samples must be bit-identical, therefore the order of floating-point operations is part of the specification* — rather than as a tip. We think it is worth a short subsection and an ablation figure in a paper, because it is a case where the naive implementation is algebraically correct and visibly wrong.

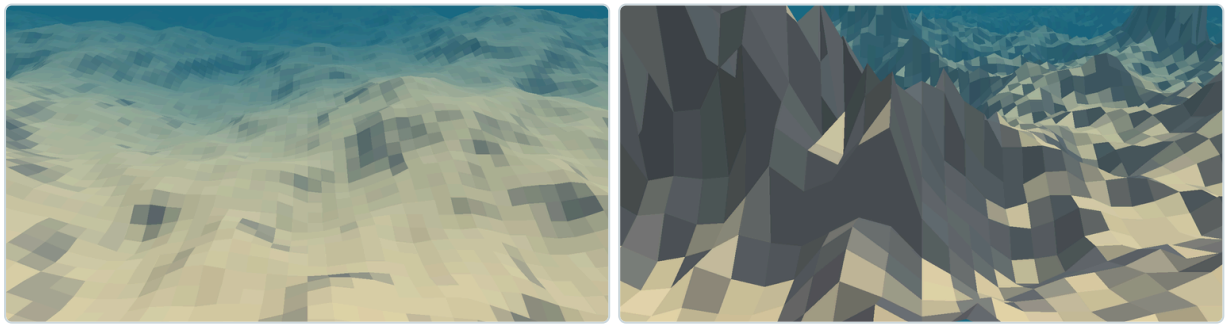

Figure 3 — The two shipping seabed styles. Left: *Classic*, rolling dunes used for the reef and kelp environments. Right: *Canyons*, the same generator with warping, ridging and terracing enabled and a taller height scale, used for the deep-water lesson. Both are single-valued height fields — every apparent cliff is still one surface per column.

3.3 A robustness note worth keeping

Our ridge operator raises a value to a fractional power. If that value is ever negative the result is not a number, and a single such value propagates through the mesh and destroys the entire chunk — it disap-

pears, or renders as an explosion of triangles. The noise function we build on can return values fractionally outside its documented range, which is enough to trigger it. We therefore clamp before folding.

It is a small thing, but it is the class of bug we would have paid dearly to find later: rare, seed-dependent, presenting as a rendering failure rather than a maths failure, and impossible to reproduce without the exact seed and chunk coordinate. We include it because "clamp before you take a fractional power of noise" is the kind of hard-won detail we think a systems paper should pass on.

4 Turning Numbers into a Surface

> **What we built:** a converter from a grid of heights into a mesh at one of four detail levels, with lighting that matches across chunk borders and no visible cracks where detail levels meet.

4.1 Why detail levels are not optional

Our chunk at full detail is a 33 × 33 grid. If we filled the visible radius at full detail we would be drawing tens of thousands of triangles of seabed that is mostly a few pixels tall on screen. On a mid-range phone that is the difference between a smooth experience and a slideshow. We therefore generate distant chunks at coarser resolutions, by simply striding the sample grid.

That forced our grid size. Striding a 32-segment grid has to divide evenly, so our usable strides are 1, 2, 4 and 8 — giving detail levels 0, 1, 2 and 4, with 3 conspicuously absent. The gap is not an oversight on our part; it is arithmetic, and we documented it in the code precisely because it looks like one.

4.2 Lighting that matches across a border

We compute a surface normal from the difference in height between a vertex's neighbours. At the edge of a chunk, one of those neighbours belongs to the *next* chunk — which may not exist yet. Both naive options were bad for us: guess the missing neighbour and lighting visibly changes across every border, or generate chunks in dependency order and lose the parallelism that makes our streaming work.

Instead we have each chunk generate one extra ring of height samples all the way around, which we use *only* for computing normals and then discard. Both chunks sharing a border compute the same normal from the same data, independently, in any order. It costs us generating a 35 × 35 array to mesh a 33 × 33 one — about 12% more arithmetic on the cheapest stage of our pipeline — and we consider that a good price for removing an entire class of visual artefact while keeping full parallelism.

4.3 Hiding the cracks instead of solving them

Where a high-detail chunk meets a low-detail one, the edges genuinely do not match — the coarse chunk has fewer vertices along the shared edge, so the fine chunk's extra vertices have nowhere to attach. The rigorous fix would have been to stitch the boundary, generating transition geometry for each combination of neighbouring detail levels. It is correct and it is fiddly.

We chose the pragmatic alternative: we drop a hidden vertical curtain from each chunk's outer edge, like the rim of a tray. Our cracks still exist, but looking through one shows more seabed rather than empty background, so in practice nobody sees them. We scale the curtain's depth with the terrain's height, because our taller canyon cliffs open wider gaps than our gentle dunes.

We want to state that plainly as a philosophy rather than dress it up: we preferred a cheap constant-cost trick that makes the artefact *unobservable* over an exact solution that would cost us implementation complexity and per-frame work. On our device budget, being invisible was the requirement; being correct was only a means to it.

Meshing, normals and skirts

★ STANDARD

We claim no novelty here. Border rings for seamless normals and skirts for LOD cracks are both well-known techniques in terrain rendering, and we used them as found. We include them because our *reasoning* transfers — trading 12% extra arithmetic for parallelism-preserving seamless lighting, and preferring unobservable over exact — and because a reader will otherwise wonder how we handled our seams and cracks.

5 Making It Endless, and Putting It in Your Room

> **What we built:** a moving window of live world around the user — generating ahead of them, coarsening in the distance, destroying behind them — while the whole world stays pinned to a real spot on a real floor.

5.1 Why streaming rather than a big world

Our window is small: a chunk is eight metres across, and our largest view distance keeps roughly sixty-four metres of ocean alive. That is not much world, and we made it not much world deliberately. Our memory stays flat whether the student walks two metres or two hundred, and we pay the generation cost a chunk at a time, in the background, rather than as a load screen.

Our streamer's whole job is bookkeeping: which chunks should exist, at what detail, which one needs a collider, and which we should destroy. We re-evaluate only when the user has actually moved a meaningful distance rather than every frame, because the answer rarely changes between frames and this check is the one piece of per-frame CPU work we have.

Colliders deserve a specific note. Physics meshes are expensive to build, and we only ever need collision near the user — for placing life on the seabed and for keeping the swimmer above it. We therefore give a collider only to the nearest detail level, and when a chunk coarsens we disable its collider rather than destroy it, so returning to it costs us nothing.

5.2 Generating without stalling the frame

Generating a chunk takes long enough that doing it during a frame would give us a visible hitch. We therefore do it on background threads — which we can only do because of the purity decision in §2.2. Our stages 1 and 2 touch no engine state, so we can safely run them anywhere.

We use a classic producer/consumer pattern: work goes out to a thread pool, finished results come back through a lock-guarded queue, and the main thread drains that queue each frame and does the small amount of work that genuinely requires the engine. We use a thread pool rather than a thread per chunk because our first version cost one operating-system thread per chunk and did not survive contact with a real phone.

> **A trap we want to pass on.** The height-remapping curve we use during meshing is an engine type that is *not* thread-safe: two threads evaluating the same curve object corrupt each other's results intermittently. Because the corruption is rare and produces plausible-looking but wrong heights, it presented to us as "occasionally a chunk is subtly misshapen" — nearly impossible to attribute. We now copy the curve per invocation. We would treat any engine object shared into a worker thread with the same suspicion.

5.3 The AR part, which is smaller than expected

Our entire AR coupling is one short component and two adaptations. The component waits for a tap, raycasts against detected planes, creates an anchor at the hit pose, instantiates the environment under that

anchor, and stops plane scanning. We gave it no knowledge of the app's tutorial, quiz, or content systems — it anchors a prefab and gets out of the way.

The two adaptations we made inside the terrain are more interesting, because they were the difference for us between "a terrain demo" and "a terrain that works in AR":

- **We take height from the anchor, but not position.** Our chunks keep their real-world horizontal position, so the world streams correctly as the user walks — but we take their vertical position from the anchor rather than from the world origin. That single decoupling is what seats our seabed on the detected floor instead of at an arbitrary height.
- **We discover the viewer rather than configure it.** In a hand-built scene you drag the camera into a slot. A prefab we instantiate at runtime into an AR scene has no such slot, so our terrain falls back to the active camera. We needed the same treatment in several components; it is a small pattern that recurs whenever scene-authored content becomes runtime-instantiated content.

Streaming, and the anchor-relative vertical decoupling ★★ COMBINATION

Chunked streaming with distance-based detail is textbook and we claim nothing for it. What we think is worth reporting is our AR adaptation: *world-space horizontally, anchor-space vertically*. It is one line of code and it was the crux of making our infinite procedural world coexist with a tracked physical floor. The procedural terrain literature we drew on assumes a fixed world origin; the AR literature rarely deals with content generated as the user moves. We found ourselves at a small but genuine point of contact between the two, and it is the kind of practical detail we think a demo-track audience actually wants.

6 Filling It with Life

> **What we built:** a way to place corals, kelp, sea fans, boulders, arches and caves on the seabed — at a density and in positions that read as an ecosystem rather than as scattered props — such that the same place always gives us the same reef.

6.1 Determinism as a persistence strategy

This was our single most consequential decision in the population stage, and it is easy to miss because it shows up as an *absence*: we have no save file.

We seed every chunk's own random number generator from a hash of its coordinates and the environment's seed. Nothing about our placement is actually random — it is a pure function of position. When we destroy a chunk as the student walks away and regenerate it as they walk back, we get exactly the same reef, in the same arrangement, at the same sizes.

The consequences for us were larger than "it looks consistent":

- **We need no persistence layer.** We have nothing to serialise, no save format to version, no save-corruption failure mode, and no storage growth as a student explores. Our infinite world costs the memory of the visible window and nothing else.
- **We get reproducibility across devices.** A teacher and thirty students holding the same environment file see the same reef. Not a similar reef — the same one. That matters when the lesson says "find the sea fan on the ridge to your left."
- **We can debug it.** We can reproduce a misplaced object exactly from its chunk coordinate, which we could not do with anything genuinely random.

> **Determinism as pedagogical reproducibility** ★★ COMBINATION
>
> We claim no novelty for the technique: hash-seeded deterministic world generation is standard in games, and it is how voxel sandbox worlds work. Our framing is what we think is worth publishing. In an *educational* context, determinism stopped being a memory optimisation for us and became a curricular guarantee — a shared environment file is a shared classroom. We would defend that reframing, plus our observation that it eliminates the persistence layer entirely, as an application contribution, and it is trivial for us to demonstrate empirically: walk away, walk back, compare.

6.2 Placement rules borrowed from ecology, not art direction

When we scattered objects randomly across the surface, the result read immediately as computer graphics. Real benthic communities are structured, and their structure follows a small number of physical rules. We implemented four of them, and we chose each so that it is a rule a marine biologist would recognise rather than a knob an artist would ask for.

Rule	What it does	Why it is ecologically right

Habitat patches	A slow, large-scale noise field gates where life may grow at all, so lush gardens alternate with open sand	Benthic cover is patchy at tens-of-metres scale, driven by substrate and current — it is not uniform
Slope limits	Each species accepts a range of seabed slopes	Sessile organisms have real attachment constraints; soft corals and sand-dwellers do not occupy the same faces
Micro-habitat by curvature	Four extra probes around a candidate point estimate whether the ground is locally convex or concave; species can be biased onto ridges and outcrop tops, or into hollows and crevice mouths	This is the strongest real signal in benthic distribution: filter feeders take elevated, current-exposed positions; cryptic species take sheltered ones
Size–frequency structure	Sizes are drawn with a bias toward the small end, so a patch is many small individuals, several mid-sized, and an occasional large one	Real populations are recruitment-dominated and follow a decreasing size–frequency distribution. Uniform sizes are the single most obvious tell of procedural placement

We added one more rule that is about honesty rather than realism: we scale every object to a specified real-world height in metres, measured from its own bounding box, regardless of how the source model was exported. A coral we specify as 30–80 cm is 30–80 cm. In an app whose purpose is teaching students what marine organisms are like, we treat scale as content rather than polish — and it is exactly the sort of thing that went silently wrong for us when models came from a dozen different sources.

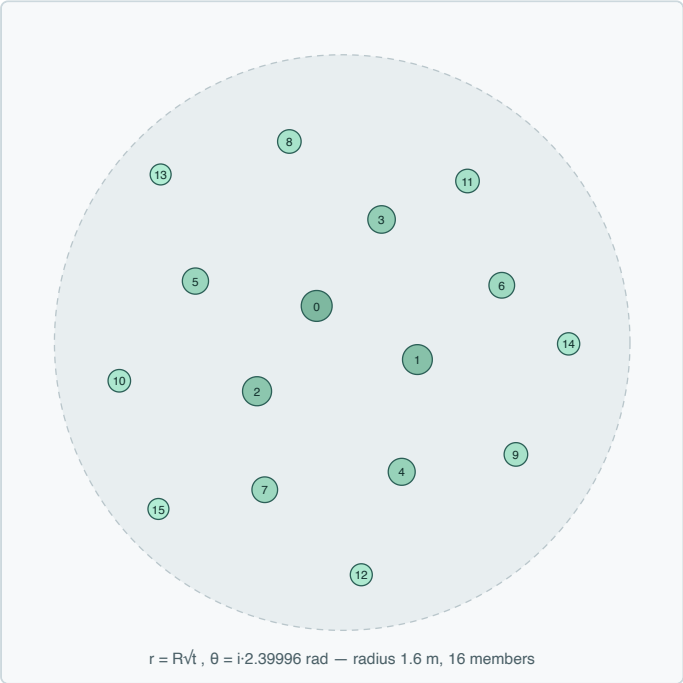

Figure 4 — How one reef mound is packed. Members are placed on a sunflower spiral: each is rotated by the golden angle from the last, at a radius proportional to the square root of its index. The square root keeps areal density even (rather than crowding the centre), and the golden angle avoids any repeating lattice. Centre members are lifted and enlarged, edge members flattened and shrunk, which gives the mound a dome profile and a size hierarchy instead of a flat disc of identical clones.

Curvature-driven micro-habitat and size–frequency structure ★★★ NEW

Slope-limited and mask-gated scattering is standard in every terrain tool, and we take no credit for those. Two of our four rules go further, and we believe they are genuinely new in this context.

Curvature-based placement. We estimate local convexity from four cheap ground probes and use it as an acceptance probability, which lets us tell a species "grow on exposed edges" or "grow in shelter" rather than merely "grow on slopes under 40°". We are not aware of a scattering system that exposes local surface curvature as an ecological placement parameter. It costs us four raycasts per candidate and we make it opt-in per species.

Size–frequency biasing. This is one line of maths with a real biological justification, and it bought us more reef plausibility than any amount of additional model variety.

Together we think these are the clearest instance of a theme worth building our paper around: *placement parameters expressed in the vocabulary of the domain being taught, not the vocabulary of the renderer.*

6.3 Solving caves without giving up the height field

In §3.1 we accepted that a height field cannot produce an overhang. But we found that a canyon lesson without a swim-through arch, and a reef without a grotto to look into, loses much of what makes exploration compelling. Our resolution was to keep the terrain a cheap height field and add true three-dimensional features as *separate meshes seated on top of it*.

We generate eight kinds of feature in code — boulder, spire, arch, overhang, grotto, kelp plant, glow anemone and sea fan. Two properties made this affordable for us on a phone:

- **A handful of meshes, shared by everything.** Each of our rules builds a small number of variants once, at startup, and every instance in the world reuses them. Our mesh memory is therefore flat in world size: an infinite ocean of arches costs us four arches.
- **No textures at all.** We flat-shade the meshes with colour baked into their vertices — including baked ambient occlusion. We *paint a grotto's interior dark at generation time*, so it reads as a cave without a single light source. It is the cheapest way we know to get interior mood on a mobile GPU.

Making rock look like rock

Our early version displaced spheres with noise, which reliably gave us something the eye reads as "a sphere that someone wobbled" — the characteristic artificial look. Real rock has three properties that a noise-displaced blob does not, and we implemented all three as one shared style:

1. **Fracture faces.** Rock splits along planes, so we project points that cross a randomly oriented plane onto it, collapsing whole patches into flat facets that meet at hard edges. This was the single change that moved our shapes furthest from "blob" toward "rock".
2. **Bedding.** We add sedimentary layers of alternating hardness: resistant bands jut out, soft bands erode back, and each hard band slightly overhangs the one below. That small shadow line is what makes stratified rock recognisable at a glance. We give the layers a few degrees of dip, because we found perfectly level bedding looks like a stack of pancakes.
3. **Detail at two scales** — we combine broad mass lumps with a fine chipped surface.

Figure 5 — From blob to rock, one property at a time. Left to right: the base ellipsoid; with two-scale noise displacement — recognisably "a wobbled ball"; with bedding strata, which introduce horizontal structure and overhanging lips; with fracture planes, which flatten patches into facets and give the silhouette hard edges. Rendered from a port of the shipping generator, flat-shaded with baked occlusion exactly as the app does it.

We bundled this as a shared *style object* rather than writing it per shape because we wanted coherence. We apply the same style to a blob (boulder), a swept tube (arch), and a hollow shell (grotto), so every rock in one of our biomes looks like the same geological material. Without it, our biomes read as an assortment of unrelated primitives — which is exactly how procedurally populated worlds usually look.

Hybrid height field + shared feature meshes, unified by one rock style ★★ COMBINATION

We are borrowing here: adding 3D props to a height field is what games do, and fracture-by-plane-projection and strata displacement both already exist in the procedural modelling literature. Two things we did are worth reporting. First, our *memory argument*: variant sharing makes feature memory constant in world size, which is the only reason caves and arches were viable inside our mobile AR streaming budget at all. Second, our *style-object* approach — one parameter set applied across topologically unrelated primitives to enforce material coherence — which is a simple idea that materially improved how our generated environments read, and which we have not seen stated as such.

7 Making It Look Like Water

> **What we built:** one description of the water — its colour, its clarity, how far you can see — that every single thing we draw reads from, so our sand, rocks, kelp, sky and surface all agree about what medium they are in.

7.1 Why one medium rather than per-object settings

We built this to prevent one failure mode: objects looking pasted onto a background. It happened to us when a rock's shader and the sky's shader disagreed slightly about the colour of distance — the rock stayed a little too crisp, or faded to a slightly different blue, and the eye immediately read it as a sticker rather than as something in the water.

So we define the water once — a handful of colours and two distances — and push it out as global values that every shader reads. We left no per-material fog setting that could drift out of sync. When a teacher picks "Coastal Green", our sand, boulders, kelp, marine snow, water surface and horizon all change together, because they are all reading the same numbers.

7.2 Colour that dies with distance, per channel

The most important single thing we did to make the medium read as *water* rather than as fog was to model the fact that water absorbs different wavelengths at very different rates. Red light is gone within a few metres; blue carries much further. This is why underwater photographs go blue-green with depth and distance, and it is why the grey haze we started with never looked wet.

Rather than ask anyone to author three extinction coefficients, we derive them from the water colour the user already chose: we absorb the channels in which the water is dark harder. A single authored colour therefore gives us physically-flavoured wavelength-dependent attenuation for free. We also made the effect's strength zero reproduce our older single-channel behaviour exactly, which is what let us introduce it without changing existing environments.

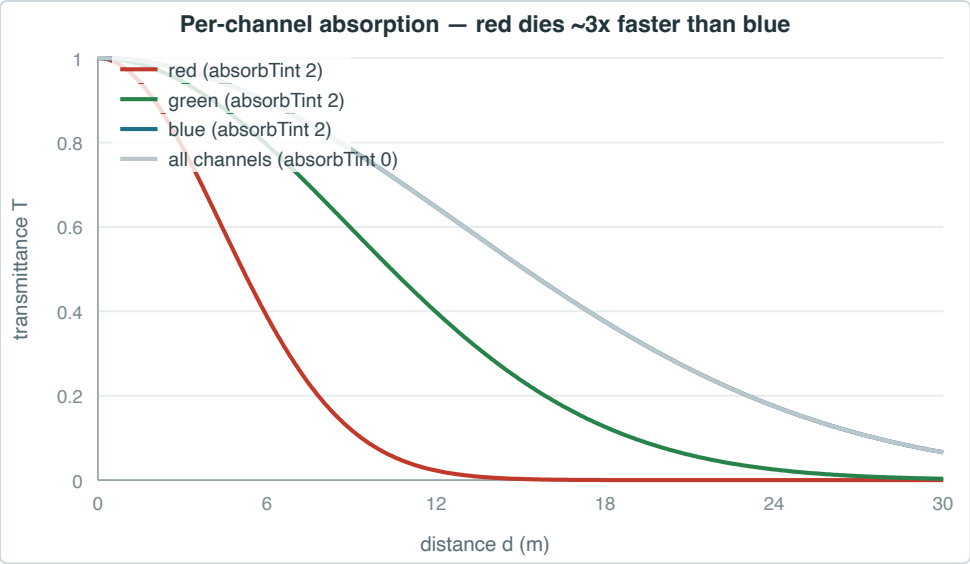

Figure 6 — Per-channel transmittance against distance, for the default tropical blue palette. Red falls away roughly three times faster than blue, so a red object twelve metres away is not merely dimmer — it is a different colour. The flat grey curve is the previous behaviour, where all three channels attenuated identically; it is still reachable by setting the effect to zero.

7.3 The coupling that makes the world have no edge

Our infinite streamed world has a boundary: the last ring of chunks, beyond which nothing exists yet. If the student can see that far, they see our world end.

The usual approach is to tune the fog until the edge is hidden, and to re-tune it whenever the view distance changes. For us that was a maintenance burden and a latent bug — a teacher who picked a large exploration area and high visibility would have brought the edge straight back.

Instead we *coupled the two by a rule*: we force the distance at which visibility reaches zero to be less than the streaming reach, with a margin. We do not recommend it as a setting — we re-derive it every time an environment is loaded or saved, so no combination of choices can violate it, and neither can editing the file directly. For us "the world never visibly ends" stopped being a tuning outcome and became a property we hold by construction.

We applied the same reasoning to tie the camera's far clipping plane to that distance — there is no reason for us to draw what is fully fogged — and to tie our marine-snow volume to it, since particles must not appear beyond the point where the water is already opaque.

Coupling the rendering medium to the streaming reach as an enforced invariant ★★★ NEW

Fogging the far plane to hide a streaming boundary is as old as streaming worlds, and we did not invent it. What we have not seen elsewhere, and what we are claiming, is treating that relationship as a *machine-enforced invariant across two independent subsystems*, re-derived on every load and save, so that no reachable configuration can expose the boundary.

We think it is the clearest single example of our general idea in §8: subsystems that must agree should be made to agree by construction, not by documentation and care. It is small, it is checkable, and we believe it generalises well beyond our application.

7.4 Cheap tricks that carry a lot of weight

Three techniques do a disproportionate amount of the work of making our scene feel underwater, and we chose all three for cost as much as for appearance:

- **Marine snow.** We found suspended particulate to be the strongest single cue that you are in water rather than air. We implement it as one static mesh of camera-facing quads whose positions we compute entirely on the GPU from a per-particle seed, drifting and wrapping around the camera. Hundreds of particles cost us one draw call and no per-frame CPU work at all.
- **Caustics.** We get the dancing light on the seabed from a small iterative distortion field evaluated per pixel, gated to upward-facing surfaces. We use no textures, no light cookies and no projectors.
- **Encrusting life on rock.** Rather than model coralline growth, we compute coverage in the shader from noise, how upward-facing the surface is, and its baked occlusion — so crust appears where light reaches and exposure allows, and not in shadowed crevices. We take its colour from the chosen habitat.

We want to mention a fourth as a correctness note rather than an effect. We sway kelp in the vertex shader, weighted so roots stay planted, and we derive the phase from world position so a whole bed moves as one current field rather than in unison. We duplicate the sway maths exactly in the depth pass — a classic source of bugs, since a plant that bends in colour but not in depth self-occludes wrongly.

> **Per-channel absorption derived from one authored colour** ★★ COMBINATION
>
> We take no credit for the physics: wavelength-dependent underwater attenuation is standard in offline and film rendering and appears in high-end real-time water. Our contribution is deriving it automatically from the single colour a non-expert already chose, so our author never sees an extinction coefficient. That is what made the feature acceptable to us under constraint C4.

8 Letting a Teacher Build One

> This is the part of our system we would build a paper around. Everything before it is our competent assembly of known technique in service of a hard set of constraints. This section is where we think we have an idea.

> **What we built:** an authoring layer in which a teacher answers ten questions, we derive about fifty technical parameters from those answers, and no combination of answers — or of hand-edited files — can produce a broken world.

8.1 Why "fewer sliders" is not the answer

Our obvious first response to constraint C4 was to expose a subset of the technical parameters: hide lacunarity, keep fog density. It did not work, for two reasons.

First, the parameters we kept were still meaningless to our user. "Fog density 0.055" is not a question a marine biology teacher has an opinion about. What they have an opinion about is whether the water is murky or clear.

Second, and more seriously: we found that individually valid settings still combine into a broken world. Every parameter can sit inside its own legal range while the environment is unusable — terrain tall enough to erupt through the water surface, or visibility long enough to see the world end. We could not fix that by restricting ranges, because the problem is not in any one range. It is in the relationships between them.

8.2 Two layers, and one direction of travel

We therefore built two distinct layers with a single translation between them.

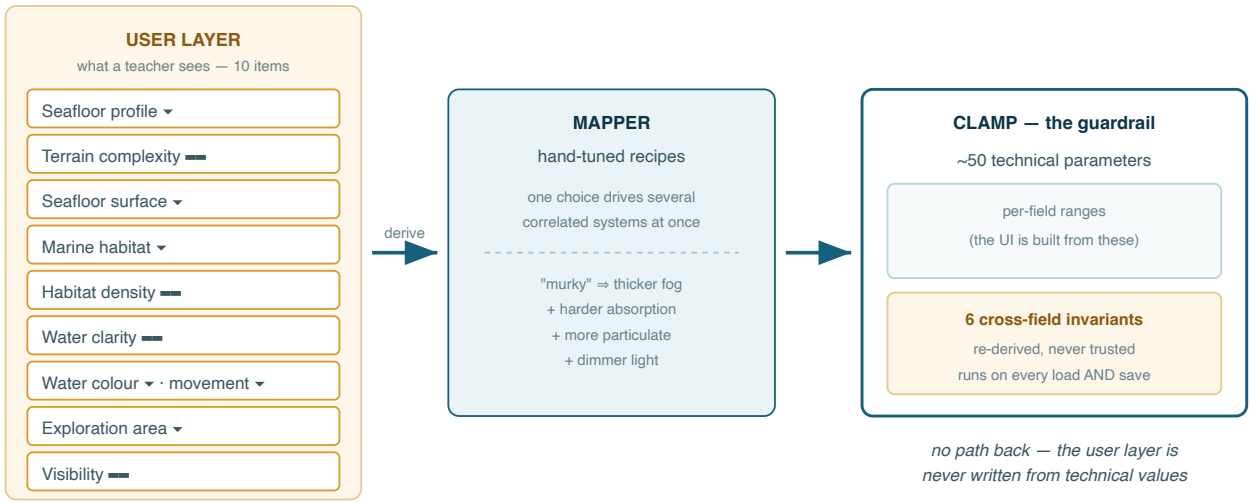

Figure 7 — The two-layer authoring model. Information flows strictly left to right. The teacher edits only the left column; our mapper expands each choice into the technical parameters it implies; our clamp enforces every range and then re-derives the cross-field invariants. Because we made the flow one-way, we can change the technical layer freely without breaking saved environments, and the user layer can never be put into a state that does not correspond to a valid world.

The mapper is where we put our domain knowledge. Each seafloor profile is a recipe we tuned by hand, and critically *one choice drives several correlated parameters at once*. Our water clarity control does not just set fog density; it also sets how hard colour is absorbed, how much suspended particulate there is, and how much light survives — because in real water those covary. We never give the user three sliders that they could set into a physically incoherent combination.

8.3 Invariants: making broken worlds unrepresentable

After every derivation we re-compute six relationships rather than trust them:

	The invariant	What it prevents
I-1	Fog must have a real fade band	A hard visibility wall instead of a gradual fade
I-2	Visibility must end before the streaming reach	Seeing the edge of the world
I-3	Terrain peaks stay below the water surface	Seabed erupting through the surface into open air
I-4	One water level drives the plane, the fog and the shaders	The visual surface and the physical surface disagreeing
I-5	The camera's far plane is derived from visibility	Drawing geometry that is already fully fogged
I-6	A hard per-chunk budget on scattered objects	A dense-habitat choice producing a frame-rate collapse

Two details make what we did stronger than validation:

We run the invariants on save as well as load. We clamp a hand-edited file, a file from an older version, or a file transferred between devices on the way in. We left no trusted path into the system.

We generate the user interface from the same range definitions that our clamp enforces. The form literally cannot express an out-of-range value, so our clamp is a second line of defence rather than the only one — and the two cannot drift apart, because we wrote only one definition.

Constrained authoring with enforced cross-field invariants ★★★ NEW

This is our strongest claim, and the one we would put in an abstract.

Procedural content tools for non-experts normally reduce risk by *restricting ranges* — the user gets a slider from 0 to 1 and whatever it produces is deemed acceptable. We found that approach cannot address failures arising from *relationships between* parameters, and in an integrated real-time system that is where the serious failures live: rendering settings that expose our streaming boundary, terrain settings that violate our water plane, density settings that break our frame budget.

We make a narrow and checkable claim: **in our system, no reachable configuration produces a world that is visually broken, physically inconsistent, or outside the performance budget** — where "reachable" includes hand-edited configuration files, not just our user interface. Our mechanism is a small set of invariants that span subsystem boundaries, re-derived at both ends of persistence.

We can also demonstrate it cheaply: enumerate the authoring space, assert the invariants hold at every point. We think that is a stronger and more interesting evaluation than a usability score, and it is the experiment we would run first.

An honest limit on our claim. By "not broken" we mean the six properties above, which are the failure modes we identified and encoded. We do not mean "attractive", and we do not mean our space contains no ugly environments — it certainly does. Our guarantee is about correctness, not quality, and we think the paper must say so in those words.

8.4 Order matters, and is enforced once

One further correctness property we needed is about sequence rather than values. We must configure the terrain generator before the streamer starts, because chunks read their parameters as we create them — a chunk built one frame early would keep default settings for its whole life. The same applies to our materials: we must swap them before any chunk or the water surface is built.

Rather than document this ordering and hope, we do all configuration in exactly one place, in one fixed order, once, at load. After that moment our environment behaves exactly like a hand-authored scene with no ongoing cost. Concentrating our entire configuration surface into a single ordered function is what lets us check the ordering constraint by reading one screen of code.

9 Letting the App Describe Itself

> **What we built:** a small read-only interface that answers, at any moment, "where is the student, what is around them, and what does it look like?" — as structured data *and* as an English sentence.

9.1 Why a generated world needs this and a hand-made one does not

In a hand-authored scene the content is known in advance. A designer can write the narration, place the quiz trigger and author the hint, because they know there is a sea fan on the ridge.

In our procedurally generated world nobody knows what is there — not us, not the teacher, and not the app. We computed the reef around the student thirty seconds ago from a hash of chunk coordinates. So any feature we want that *talks about* the environment — spoken narration for a visually impaired student, a comprehension question about what is nearby, an AI tutor answering "what is that?" — needs a way to ask the world what it currently contains.

We think this is a real and slightly under-appreciated cost of procedural generation in an educational setting, and it is why we made this interface a first-class part of the subsystem rather than a utility.

9.2 What it reports

We answer in the vocabulary a person would use, not the vocabulary the engine stores:

- **Where the student is** — depth below the surface, compass heading, and the distance to the seabed, which we measure by casting a ray downward.
- **What kind of place it is** — a depth band we name in domain terms: sunlit shallows, reef zone, open midwater, deep seabed, or close to the seabed.
- **What the water is like** — light level, visibility in metres and a colour description, which we read from the same medium settings that drive our shaders, so our description cannot disagree with what is on screen.
- **What is nearby** — creatures and features within a radius, each with a distance, a *relative* bearing ("ahead-left", "behind") and a vertical relation ("above you", "same depth"), which we sort nearest-first and cap.

The bearings are our important design decision here. Reporting world coordinates would have been useless to a narrator, whereas "ahead-left, four metres, above you" is directly speakable and directly actionable. We emit the snapshot both as machine-readable data and as an assembled English summary, so a text-to-speech feature can read it directly while a model-driven feature can consume the structure.

A scene-state-to-language interface for a procedurally generated lesson ★★ COMBINATION

We are not first here: scene description for accessibility is an active area, and exposing game state to a language model is now common practice. Our specific framing is what we offer. Because we generate the environment rather than author it, *no other part of our application can know what the lesson contains*, so a self-description interface became a structural requirement for us rather than an accessibility add-on. We also found it to be the natural attachment point for spoken narration, generated assessment and an AI tutor — three things the wider application wants — from a single small read-only surface.

A defect we should state. Our snapshot reports a water temperature that is always zero: we wrote the estimator and the summary reads the field, but we never assign it. It is a one-line fix and we should make it before demonstrating or writing up this interface, because temperature is exactly the sort of value a marine biology narration would use.

10 The Novelty Ledger

> We collect every claim we make here, graded. Read our ★★★ rows as the contribution, our ★★ rows as the systems argument, and our ★ rows as evidence that we know which parts are not ours.

10.1 Grade ★★★ — the claims we would defend

Claim	Why it is new	How to demonstrate it
Constrained authoring with enforced cross-field invariants §8.3	PCG authoring tools reduce risk by restricting each parameter's range. That cannot prevent failures caused by *relationships between* parameters across subsystems. We encode six such relationships and re-derive them at both ends of persistence, so no reachable configuration — including hand-edited files — yields a broken world	Enumerate the authoring space and assert all six invariants at every point. A machine-checkable result, far stronger than a satisfaction score
Coupling the rendering medium to the streaming reach §7.3	Fogging a streaming boundary is old; making the relationship a machine-enforced invariant across two independent subsystems, so the boundary *cannot* be exposed by any setting, is not something we have seen stated	Set visibility and exploration area to their extremes and show the boundary never appears; show the derived value being clamped
Curvature-driven micro-habitat placement §6.2	Scatterers universally support slope and masks. Estimating *local surface convexity* from four cheap probes and exposing it as an ecological placement parameter — "grow on exposed ridges", "grow in sheltered hollows" — appears to be new, and matches the strongest real driver of benthic distribution	Side-by-side scatter of the same species with the parameter negative, zero and positive
Placement expressed in the domain's vocabulary §6.2	Species are described by slope tolerance, exposure preference, real-world size in metres and a size–frequency distribution — the terms a marine biologist uses — rather than by density, jitter and scale multipliers	Present the authoring vocabulary beside a conventional scatter tool's parameter list

10.2 Grade ★★ — known parts, our assembly or framing

Claim	The argument
The four-constraint conjunction §1.2	Infinite streamed PCG, in AR, on a phone, authored by a non-developer, with a correctness guarantee. Each has prior work; the combination and its interactions do not
Determinism as pedagogical reproducibility §6.1	Hash-seeded generation is standard practice. Reframing it as a curricular guarantee — one file means thirty students see the same reef — and noting that it removes the persistence layer entirely, is the contribution
Hybrid height field + shared feature meshes §6.3	The memory argument is what matters: variant sharing makes feature memory constant in world size, which is what makes caves and arches affordable inside a mobile AR budget
One rock style across unrelated primitives §6.3	Fracture and bedding both exist in the literature. Applying one parameter set across a blob, a swept tube and a hollow shell to enforce material coherence is a simple idea with a large effect on how a generated biome reads
Per-channel absorption from one authored colour §7.2	Wavelength-dependent attenuation is standard in offline rendering. Deriving it from the single colour a non-expert already chose is what made it usable under constraint C4
Anchor-relative vertical decoupling §5.3	World-space horizontally, anchor-space vertically: one line, and the crux of making an infinite generated world sit on a tracked physical floor
Seam-safe coordinate grouping, documented §3.2	Practitioner folklore, stated as a correctness property with its floating-point cause named. The naive implementation is algebraically correct and visibly wrong
Scene-state-to-language for a generated lesson §9	Because the world is generated, self-description is structural rather than optional — and it is the single attachment point for narration, assessment and an AI tutor

10.3 Grade ★ — standard technique, and we say so

We consider the following established, and we cite them rather than claim them. We are not listing them out of modesty; we list them because it is what makes the rows above credible.

Technique	Where it appears
Fractal noise (fBm), domain warping, ridged noise, terracing	§3.1 — the seabed generator
Chunked streaming with distance-based level of detail	§5.1
Border rings for seamless normals; skirts for LOD cracks	§4.2, §4.3
Thread-pool producer/consumer with main-thread dispatch	§5.2

Sunflower (Vogel) packing for even areal distribution	§6.2, Figure 4
Icosphere subdivision; flat shading by vertex duplication; baked vertex occlusion	§6.3
GPU-computed billboard particles	§7.4 — marine snow
Vertex-shader foliage animation; analytic wave normals	§7.4
Staged generate–mesh–stream–populate pipeline with a pure/impure split	§2

10.4 If we only make one claim

We would make it §8.3. It is our most defensible claim, the easiest for us to demonstrate, the least dependent on taste, and the most transferable to other people's work. In a single sentence:

> Procedural authoring tools for non-experts protect users by restricting what each parameter may be. We show that the failures that matter in an integrated real-time system come from **relationships between parameters that span subsystems**, and that encoding a small number of such relationships as invariants — re-derived on every load and save — makes broken worlds unrepresentable rather than merely unlikely.

Our marine biology application then becomes the setting that motivates and validates the idea, rather than the contribution itself. We think that is a stronger paper than one in which we claim a procedural ocean.